

MULTITRACK_WORKTREE_RUNBOOK

Multi-Track Worktree Runbook

Revision: 1 Last modified: 2026-07-11T00:00:00Z

Purpose

This runbook documents the multi-track parallel-development layer for `helix_code`, as it is actually wired in this checkout today: the per-host track topology, how an operator brings the tracks up, the `/mnt/track<N>` mount dependency and its safety boundary, and the work-claim / work-binding / device-lock / supervisor sub-systems that coordinate parallel work across tracks.

The engine itself is **not** owned by `helix_code` — it lives in the constitution submodule at `constitution/scripts/multitrack/` and is consumed **by reference** (never copied) per §11.4.28(B) / §11.4.177 / §11.4.187. `helix_code` supplies only the per-host data file at `config/multitrack/<hostname>.yaml`. Everything below is a description of what the scripts in that directory actually do when driven with this project’s config, not a design proposal.

This document is referenced from: - `config/multitrack/the-factory.yaml` (the per-host config, line 36) - `constitution/scripts/multitrack/multitrack.sh` (its own up / status output and `--help` text) - `constitution/scripts/multitrack/multitrack_bootstrap.sh` (the operator-hand-off note printed when a track’s drive is not mounted)

1. Track topology

The per-host config for this host is `config/multitrack/the-factory.yaml`. It declares four tracks:

Track	Role	Branch	Mount	Focus
track-1	main	feature/helixllm-full-extension	/mnt/track1	current working items; the conductor (this session) coordinates from /

Track	Role	Branch	Mount	Focus
				home, and track-1/ main is never worktree-bound
track-2	feature	feature/* (branch pattern)	/mnt/ track2	owned-submodule / feature work-stream A
track-3	feature	feature/* (branch pattern)	/mnt/ track3	owned-submodule / feature work-stream B
track-4	feature	feature/* (branch pattern)	/mnt/ track4	owned-submodule / feature work-stream C

Each track entry also carries a `drive_serial` (the physical NVMe device backing that mount) and `fs: btrfs`.

Why track-1/main is never worktree-bound: git refuses to check out the same branch into two working trees at once. Since main’s branch (feature/helixllm-full-extension on this host) is also the branch the conductor’s own /home checkout is on, track-1 cannot be a second worktree of that same branch — the config’s focus field for track-1 says exactly this (“conductor (claudio2) coordinates from /home, track-1/main never worktree-bound”). Track-1’s row in `multitrack.sh status / up` therefore reports on the **drive mount state only**, not on a worktree living at `/mnt/track1`.

The `worktree_subdir` key (`helix_code` in this config) is the subdirectory name created under each *feature*-role track’s mount — i.e. a feature-track worktree checkout lives at `/mnt/track<N>/helix_code`. Per §11.4.178’s multi-project-per-track clarification, a `/mnt/track<N>` mount is **not** exclusive to one project — other projects (e.g. ATMOSphere, `helix_ota`) may have their own subdirectories under the same mount; `helix_code` only ever creates and touches its own `helix_code` subdirectory.

helix_code has no physical test-device pool (device_pool: [], lease_policy: {} in the config) — unlike hardware-testing projects that use this same engine, there is nothing for the device-lock arbiter (§3 below) to lease on this host for this project; that sub-system is present because the engine is shared, not because helix_code uses it today.

2. Bringing tracks up: multitrack.sh status / up

The entrypoint is constitution/scripts/multitrack/multitrack.sh, which binds together multitrack_config.sh (host/config resolution), multitrack_registry.sh (the .ws_state/streams.tsv state accessor), and multitrack_device_lock.sh (the device-lock arbiter described in §3) into one command surface. It does **not** reimplement any of those three — it only probes and reports.

```
bash constitution/scripts/multitrack/multitrack.sh status
bash constitution/scripts/multitrack/multitrack.sh up
bash constitution/scripts/multitrack/multitrack.sh help
```

status is a pure, read-only report: for every configured track it (a) reads the registry's recorded mount_state and ws_state via multitrack_registry.sh get, (b) does a live, read-only mount probe against /proc/self/mounts (falling back to /proc/mounts) to see whether that track's mountpoint is *actually* mounted right now, and (c) prints a per-track table plus a device-lock pool section (via multitrack_device_lock.sh status). It always exits 0 — a blocked track is *data in the report*, never a failure of the status command itself.

up is the preflight step: for every track it re-runs the same live mount probe. If the mount **is** live but the registry disagrees (mount_state != mounted), it reconciles the registry via multitrack_registry.sh set <id> mount_state mounted — this is the *only* write up performs, and it is a no-op when the registry already agrees with reality. If the mount is **not** live, up prints NOT-MOUNTED (registry=...) -> OPERATOR-BLOCKED for that track and increments a blocked counter; it does **not** attempt to mount, format, or otherwise touch the drive.

Exit codes for up: - 0 — every configured track is live-mounted (“All tracks mounted — the multi-track layer is UP”). - 20 — one or more tracks are not mounted. This is the OPBLOCKED_RC constant in the script; multitrack.sh prints the §11.4.21 operator-block note (reproduced in §4 below) and returns 20 rather than treating the unmounted state as an internal error. - 2 — usage error (no/unknown subcommand). - 1 — internal error (a sibling script is missing, or the per-host config cannot be resolved/parsed at all).

Neither status nor up ever mounts, unmounts, or formats a physical drive under any circumstance — see §4.

3. Work-claim, work-binding, device-lock, and the supervisor

These four sub-systems are independent, composable pieces the engine provides for coordinating parallel tracks. `helix_code` on this host uses the work-claim and work-binding pieces (its work is done across tracks); the device-lock piece is present but has an empty pool for this project (§1); the supervisor is the ruler's own crash-resilience layer.

3.1 Work-claim registry (`multitrack_claim.sh`) — §11.4.176-A

Guarantees **exactly-once** assignment of a workable item (or a logical group of items) to at most one track at a time, so that two parallel tracks never grab and duplicate/clobber the same work.

```
bash constitution/scripts/multitrack/multitrack_claim.sh claim
    <item-id> <track> [--ttl SEC] [--pid PID]
bash constitution/scripts/multitrack/multitrack_claim.sh release
    <item-id> [--track <id>]
bash constitution/scripts/multitrack/multitrack_claim.sh owner
    <item-id>
bash constitution/scripts/multitrack/multitrack_claim.sh status
bash constitution/scripts/multitrack/multitrack_claim.sh reap
bash constitution/scripts/multitrack/multitrack_claim.sh
    reconcile
```

Mechanically: `claim` is a check-and-claim performed inside one flock critical section — it succeeds if the item is free, is idempotent if the *same* track re-claims it (refreshing the TTL), and is refused with exit 3 (EBUSY) if a *different* track already holds it. Every claim carries an expiry (default 86400s / 24h via `MT_CLAIM_TTL`); a crashed track's claim auto-expires (`reap`) rather than being held hostage forever, per §11.4.147.

State lives under `.ws_state/`: `claims.jsonl` is an append-only event log (CLAIM / RELEASE / REAP / DENY — never rewritten), `claims.snapshot` is the current-claims table (atomically rewritten, temp-then-rename), and `claims.lock` is the flock target.

3.2 Work-to-track/branch binding (**multitrack_work_binding.sh**) — §11.4.191

Answers “does this changed-file set (or this ticket) belong to a logic-group whose canonical branch/track is *not* the current checkout?” It is a read-only resolver — sqlite3 queries against the workable-items DB plus read-only git plumbing, no writes, no locks — consumed by both the preventive PreToolUse guard hook (guard-work-track-binding.sh) and the detective commit-wrapper / pre-build gate.

```
bash constitution/scripts/multitrack/multitrack_work_binding.sh
    resolve --staged
bash constitution/scripts/multitrack/multitrack_work_binding.sh
    check --staged
```

resolve maps each input (a file path, or a ticket via --ticket ATM-NNN) to its owning logic-group via the group_paths file-scope manifest (longest-glob-wins) or via items.logic_group, then prints that group’s authoritative destination branch and canonical track. check --staged derives the current branch (git rev-parse --abbrev-ref HEAD), the current track (from the /mnt/track<N> prefix of the cwd), and the changed-file set, and exits 2 (BLOCK, reason on stderr) if any matched input’s group destination disagrees with where the change is actually landing. An unreadable registry is a **fail-closed** 2, never a silent allow (§11.4.6). Unclassified inputs (no glob owner, no ticket group) are treated as main-eligible and skipped — an honest partial-coverage gap, not a false block.

3.3 Device lock (**multitrack_device_lock.sh**) — §11.4.119 / §11.4.176-B

Arbitrates exclusive use of a shared, scarce device pool across tracks, keyed by the device_pool / lease_policy entries in the per-host config. For helix_code that pool is **empty** (§1) so this sub-system currently has nothing to lease on this host; the commands below are documented for completeness since the arbiter is shared engine code:

```
bash constitution/scripts/multitrack/multitrack_device_lock.sh
    pool
bash constitution/scripts/multitrack/multitrack_device_lock.sh
    status
bash constitution/scripts/multitrack/multitrack_device_lock.sh
    acquire --track <id> --caps <c1,c2> [--count N] [--ttl
        SEC]
bash constitution/scripts/multitrack/multitrack_device_lock.sh
    heartbeat --track <id>
bash constitution/scripts/multitrack/multitrack_device_lock.sh
    release --track <id> [--device <id>]
```

The arbiter is deadlock-free by construction: multi-device acquire is all-or-nothing inside one critical section (no hold-and-wait), it never blocks — it either wins the whole requested set immediately or returns EBUSY (exit 3) at once (no circular-wait), and a crashed holder’s lease auto-expires via TTL + reap (relaxed no-preemption). State lives under MT_LOCK_DIR (default `${XDG_RUNTIME_DIR:-/tmp}/<project>/multitrack/devicelock`, i.e. ephemeral tmpfs-class storage — correct, since a lease has no meaning across a host reboot).

3.4 Ruler self-supervisor (`multitrack_supervisor.sh`) — §11.4.187 / §11.4.147

The multi-track *ruler* (the conductor session driving all tracks) is itself made crash-resilient: it periodically writes a durable `ruler_state.json` recording, per track, the bound alias / session id / worktree / last verdict, and a lightweight watchdog can detect a dead ruler PID and rehydrate every track’s known binding.

```
sh constitution/scripts/multitrack/multitrack_supervisor.sh
    snapshot [--ruler-pid PID]
sh constitution/scripts/multitrack/multitrack_supervisor.sh
    watch [--once] [--interval SECONDS] [--ruler-pid PID]
sh constitution/scripts/multitrack/multitrack_supervisor.sh
    status
```

`snapshot` reads the live, ephemeral session registry (`multitrack_sessions.sh`’s `tracks/<track>.json` files, falling back to replaying `events.jsonl` when a per-track file is itself missing) and merges it into the durable state file, written atomically (write-temp-then-rename). `watch` checks whether the recorded `ruler_pid` is still alive (`kill -0`); if it is dead (or no prior state exists), it rehydrates by re-running the same merge and claiming a fresh `ruler_pid`. The durable state directory resolves to `<repo-root>/ .ws_state/multitrack` — deliberately **never** `${XDG_RUNTIME_DIR}` or a hardcoded `/tmp` path, since the whole point is to survive the loss of the ephemeral session registry.

`multitrack_bootstrap.sh` (the single idempotent wiring command — see §5 below) can optionally start this watchdog as a backgrounded daemon via `MT_SUPERVISOR_WATCH_ENABLE=1`; it is **off by default**.

3.5 Host-budget guard (`multitrack_host_budget.sh`)

A sourced (not executed) pure-function library the ruler’s spawn/launch layer calls **before** starting any per-track worker process. It answers “is it safe to start one more worker right now?” by checking three independent conditions, all of which must hold:

1. **Pool cap** — live worker count < MT_MAX_WORKERS (default 4, one per `/mnt/track<N>`). There is no `--force` / no-cap escape anywhere in the file.

2. **Build-quiet** — no heavy build (m -j / gradle / the JVM compile-daemon family) is currently in flight, per the same concurrency-hardening precedent as CLAUDE.md §12.7/§12.8.
3. **§12.6 budget** — the *projected* aggregate RSS of (live workers + one more) stays inside the 60%-of-RAM ceiling, using the same `host_safe_parallel_jobs()` arithmetic the build path already uses.

```
. constitution/scripts/multitrack/multitrack_host_budget.sh
if mt_host_budget_can_spawn; then
    : # proceed to spawn one more worker
fi
```

Exit codes from `mt_host_budget_can_spawn`: 0 allow, 1 refuse (pool cap), 2 refuse (heavy build in flight), 3 refuse (would exceed the memory budget), 4 refuse (the host-safety library could not be resolved/sourced — an honest degrade, never a fake-pass). A human-readable reason line is always printed on stdout.

4. The `/mnt/track<N>` mount dependency — read this before touching a drive

These drives are ATMOSphere-owned, LUKS2-encrypted btrfs volumes.

`helix_code` coexists on the same track-parent mounts as other projects (ATMOSphere, `helix_ota`) per §11.4.178's multi-project-per-track model; it only ever creates and uses its own `helix_code` subdirectory under an already-live mount.

`helix_code` — and every script in `constitution/scripts/multitrack/` that this runbook documents — NEVER mounts, unmounts, formats, or partitions these drives. This is not a convenience choice; it is a hard constitutional boundary (§11.4.133: changes to the target system and its hardware must always be safe, and an autonomous agent must never perform an irreversible, high-blast-radius storage operation it cannot verify is safe). `multitrack.sh` up's own operator-block note states this explicitly:

mount each track's LUKS2+btrfs NVMe drive at its `/mnt/track<N>`. ... mounting needs `su` + the LUKS passphrase, which this agent does not hold, and §11.4.133 forbids autonomous mount/format of a physical drive. This is NOT a failure — it is a bounded operator hand-off (§11.4.101).

Operator unblock procedure, when `multitrack.sh status` or `up` reports a track as `unmounted-locked` / `OPERATOR-BLOCKED`:

1. The operator (with `su/root` on this host and the LUKS passphrase) mounts the affected track's encrypted `btrfs` volume at its configured mountpoint (`/mnt/track1 .. /mnt/track4`).
2. If `helix_code`'s own project subdirectory does not yet exist under that mount (a fresh drive, or a freshly-mounted track this project has not used before), create it as `/mnt/track<N>/helix_code` — a plain directory create, never a partition/format operation — and populate it as a git worktree/checkout of this project (see the `worktree_subdir` config key in §1).
3. Re-run `bash constitution/scripts/multitrack/multitrack.sh up`. A now-live mount reconciles the registry automatically (§2) and the track moves from `OPERATOR-BLOCKED` to `ACTIVE`.

No script documented in this runbook will do steps 1 or 2 on its own, under any environment variable or flag. `multitrack_bootstrap.sh`'s own drive-prep step (§5, step 5) is explicitly read-only/informational for exactly this reason — it reports which tracks need an operator mount, and stops there.

5. Bootstrap (for reference)

`multitrack_bootstrap.sh` is the one idempotent command that wires the whole engine into a project out of the box (installs the toolkit `cwd-hook` symlink, validates the per-host config, seeds/reconciles the orchestrator's alias ↔ track bindings, and reports — never acts on — any unmounted track). It is invoked automatically by the constitution submodule's own auto-propagation seam on every constitution pull, and can also be run directly:

```
bash constitution/scripts/multitrack/multitrack_bootstrap.sh  
[PROJECT_ROOT]
```

Exit codes: 0 fully wired (or wired-except-informational-notes), 1 a required sibling script is missing or `cwd-hook` install failed, 2 no per-host config resolvable for this host (the genuinely fatal, §11.4.6 path — it prints the exact YAML template + the authoring step rather than inventing a default), 3 config resolved but failed to parse or had zero tracks, 4 orchestrator reconcile/status failed unexpectedly.

Known documentation gap

`multitrack.sh`'s own header and help text also reference `docs/guides/MULTITRACK_ACTIVATION.md`. As of this writing that companion document does not exist in this checkout — this is a pre-existing gap this runbook does not close (out of scope for this PWU); it is flagged here rather than silently ignored, per §11.4.6. Likewise, none of the twelve-plus scripts under `constitution/scripts/`

multitrack/ yet have a docs/scripts/*.md companion per §11.4.18 — the supervisor and host-budget scripts' own header blocks note this as a pre-existing gap, not a regression introduced here.

**Sources verified 2026-07-11: constitution/scripts/
multitrack/multitrack.sh, constitution/scripts/
multitrack/multitrack-up, constitution/scripts/
multitrack/multitrack_bootstrap.sh, constitution/
scripts/multitrack/multitrack_claim.sh,
constitution/scripts/multitrack/
multitrack_work_binding.sh, constitution/scripts/
multitrack/multitrack_device_lock.sh, constitution/
scripts/multitrack/multitrack_supervisor.sh,
constitution/scripts/multitrack/
multitrack_resolve_worktree.sh, constitution/
scripts/multitrack/multitrack_host_budget.sh,
config/multitrack/the-factory.yaml**